Lecture I: Optimal Stopping

Programming: Everyday Decision-Making Algorithms

LECTURER

Dr. Nils Roemer, External Lecturer

AFFILIATION

Planningio GmbH

Today in one sentence

When options come one by one and you can’t go back, look at the first 37 % without committing, then take the first one that beats them all.

WHY IT MATTERS

You visit flats in Hamburg. Each viewing, you must decide on the spot, and if you say no, the flat is gone. Choose too early and you miss better ones; wait too long and the best is behind you. Companies face the same problem when hiring, choosing suppliers or accepting offers.

Roadmap

01 The problem: choosing the best when options come one at a time

02 The rule: look first, then leap

03 Real life: rejection and time pressure

Options come one by one, and there is no going back

Question: Anybody know what optimal stopping is?

- **Optimal stopping** is the problem of choosing the **best option**
- from a **sequence of options**
- that are revealed **one by one**

Speaker notes

Topic: understanding optimal stopping problems and the famous Secretary Problem. Why this matters: optimal stopping is everywhere in life, from hiring decisions to finding apartments to choosing when to sell stocks. Today we learn the mathematical strategy that maximizes the chance of making the best choice.

Learning objectives: define optimal stopping problems and identify them in real-world scenarios; apply the 37 % rule (look-and-leap strategy) to make better decisions; connect optimal stopping principles to programming logic and algorithmic thinking.

Old agenda: what is optimal stopping (definition and examples), the Secretary Problem (classic formulation and solution), the 37 % rule (why it works and how to apply it), variations and extensions (rejection, time constraints, other scenarios), connecting to programming.

Anybody an example of optimal stopping?

Flat Hunting



Hiring applicants



Dating



The Secretary Problem: hire now or keep looking?

- You are **hiring a secretary** and interview candidates **one by one**
- After each interview: **hire or continue searching**
- Once you reject a candidate, **you cannot go back**
- How to **maximize the chance of hiring the best candidate?**

Speaker notes

The name comes from the 1960s. The problem is about finding the best candidate, not about secretaries. It is also known as the marriage problem or the dowry problem.

The setup in four lines

- We have `n` candidates and interview them **one by one**
- After each one: **hire or continue searching**
- We can only **rank** candidates against each other (**ordinal ranking**)

Question: Anybody know what ordinal ranking is?

Speaker notes

Ordinal ranking: we can say which of two candidates is better, but we do not have a score or an absolute scale. This is what makes the problem hard, we never know how good the best candidate could be.

Two ways to fail

Question: Anybody an idea how we can fail?

1. Reject everyone and never hire: **stopping too late**
2. Hire someone too early: **stopping too early**

Ideas?

The problem: choosing the best when options come one at a time

WHAT WE COVERED

Many decisions share one structure: options arrive one at a time, and there is no going back.

Roadmap

01 The problem: choosing the best when options come one at a time

02 **The rule: look first, then leap**

03 Real life: rejection and time pressure

Look at the first 37 %, then leap

1. **Look** at the **first 37%** of candidates without hiring anyone
2. **Remember** the best candidate so far
3. **Leap:** hire the next candidate who is **better than that one**
4. This finds the very best candidate in **37 %** of all cases

Speaker notes

This is the mathematically optimal strategy. It means we still fail 63 % of the time, but this is the best we can possibly do. The 37 % holds for a large number of candidates; with only a few candidates, the chance is even higher.

Picking at random gets worse with every candidate

Chance of the best candidate if we **pick one at random**:

- 1 candidate: 100%, 2 candidates: 50%, 3 candidates: 33%
- 4 candidates: 25%, 5 candidates: 20%

Question: Do you see the pattern?

Pattern: 1/n. Look-and-leap stays at **37 %** for any number of candidates.

Speaker notes

Random choice gets worse with every extra candidate, because each individual candidate is less likely to be the best. The look-and-leap strategy does not have this problem: its success chance converges to 37 % as the number of candidates grows.

Why 37%?

- Look at the first share x of candidates, then leap
- Chance of getting the best: about $-x \ln x$
- In words: look longer and your benchmark improves, but the best may already be gone
- Largest at $x = 1/e \approx 37\%$[1]

1. e is the base of the natural logarithm (≈ 2.718). The math is more advanced than we need here; see Ferguson, T.S. (1989) 'Who solved the secretary problem?', Statistical Science, 4(3). doi:10.1214/ss/1177012493.

Speaker notes

The formula balances two risks. If we look at too few candidates, our benchmark is weak and we leap on a mediocre candidate. If we look at too many, the best candidate is likely among those we have already rejected. The product of “benchmark is good” and “best is still ahead” peaks at 1/e of the candidates, so the optimal stopping point is at n/e.

The computer does the math for us

```python
import math

percentage = 1/math.e
print(f"Percentage of options to look at: {percentage:.1%}")

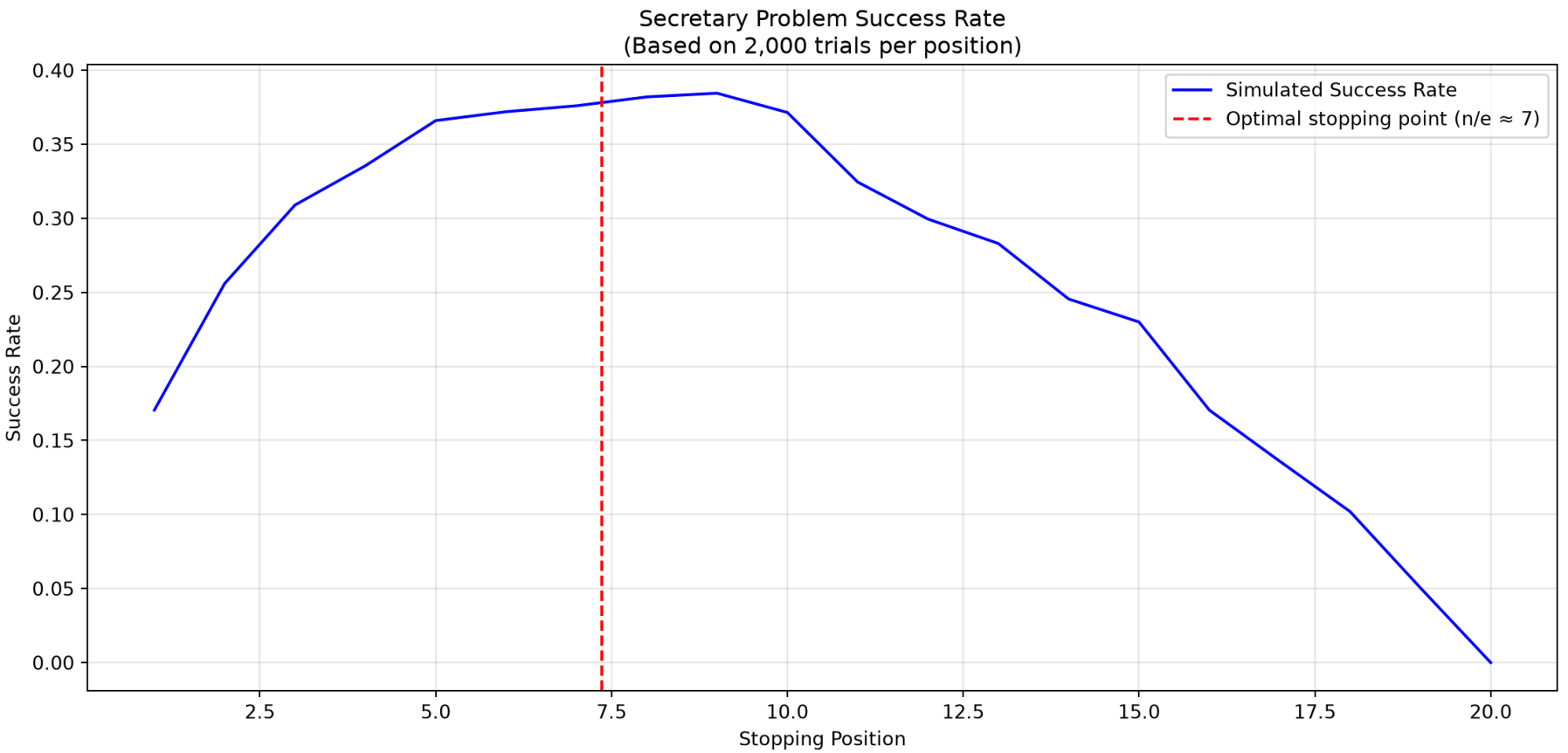
candidates = 20
lookout_phase = candidates/math.e
print(f"Look at first {lookout_phase:.3f} candidates")
```

```
Percentage of options to look at: 36.8%
Look at first 7.358 candidates
```

ⓘ No worries if you don't understand the code yet! It just applies the formula.

Simulation: looking at 7 of 20 candidates works best

20 candidates, 2,000 simulated runs per stopping point:



The rule: look first, then leap

WHAT WE COVERED

Looking at the first 37 % and then taking the first better option gives the best possible chance of picking the very best: 37 %.

Roadmap

01 The problem: choosing the best when options come one at a time

02 The rule: look first, then leap

03 **Real life: rejection and time pressure**

If you can be rejected, leap earlier

Question: In dating, **the other person can also say no**. **When to leap?**

- 50% chance of rejection: **leap at 25%**
- **Formula:** $q^{\frac{1}{1-q}}$, q = chance that your offer is **accepted**
- In words: the more often you hear “no”, the earlier you leap

Speaker notes

Life lesson: higher risk of rejection means we should be less picky. With q = 0.5 the formula gives 0.5 to the power of 2, that is 25 %. If offers are always accepted (q close to 1), the formula gives back the familiar 1/e.

Fixed time instead of a fixed number of options

Example: one year to find a flat, viewings come as they come

Question: How should we adapt our strategy?

- **Same principle:** look for the first 37% of the time, then leap
- Now we also control **how hard we search**

Speaker notes

This holds if options arrive evenly over time. We can also decide how intensively we search, for example by viewing more flats per week. Searching harder in the look phase improves the benchmark.

The same idea in other places

- Selling a house for the **best price** (“Threshold Rule”)
- A burglar deciding **when to stop** (“Burglar’s Problem”)
- **Finding** a parking spot (“Parking Lot Problem”)

Speaker notes

Side note for drivers: an increase in occupancy from 90 to 95 % doubles the search time for all drivers. Each variation changes the rule a little, but the structure stays: options come one at a time, and you cannot go back.

Real life: rejection and time pressure

WHAT WE COVERED

If you can be rejected or time is short, start leaping earlier.

What we covered

THE PROBLEM: CHOOSING THE BEST WHEN OPTIONS COME ONE AT A TIME

Many decisions share one structure: options arrive one at a time, and there is no going back.

THE RULE: LOOK FIRST, THEN LEAP

Looking at the first 37 % and then taking the first better option gives the best possible chance of picking the very best: 37 %.

REAL LIFE: REJECTION AND TIME PRESSURE

If you can be rejected or time is short, start leaping earlier.

Why it matters

You visit flats in Hamburg. Each viewing, you must decide on the spot, and if you say no, the flat is gone. Choose too early and you miss better ones; wait too long and the best is behind you. Companies face the same problem when hiring, choosing suppliers or accepting offers.

Look first.
Then leap.

Any questions so far?

After the break: First Steps in Python

- A gentle introduction to Python in our notebooks
- We turn today's idea into **code and experiments**
- This is how **algorithmic thinking** starts: break a decision into logical steps

Speaker notes

How does optimal stopping connect to programming? Algorithmic thinking: break complex decisions into logical steps. Trade-off analysis: exploration vs. exploitation, fundamental in AI and the topic of the next lecture. Mathematical optimization: using formulas to find the best solution.

Key takeaways: optimal stopping problems are everywhere in life and business; the 37 % rule is a mathematically optimal strategy; exploration vs. exploitation is a fundamental trade-off; real-world variations require adapting the basic strategy; this thinking builds the foundation for algorithmic problem-solving.

That’s it for optimal stopping! Let’s have a short break and then continue with our first Python programming session.

Literature

Interesting literature to start

- Christian, B., & Griffiths, T. (2016). Algorithms to live by: the computer science of human decisions. First international edition. New York, Henry Holt and Company.[1]

1. The main inspiration for this lecture. We have read it and discussed it in depth, always wanting to translate it into a course.

Optimal Stopping in Depth

- Ferguson, T.S. (1989). Who solved the secretary problem? Statistical Science, 4(3). doi:10.1214/ss/1177012493.

Books on Programming

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. Free online
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.



Think Python is free online and a great start. Schrödinger programmiert Python is a playful alternative in German.

More Literature

For more interesting literature, take a look at the literature list of this course.